Engine de traitement et d'analyse de flux de données sécurisé

Rapport de projet final — Python avancé

12,1 Go	69 294 532	210 834	48
de logs analysés	lignes lues	alertes de sécurité	IP attaquantes ayant compromis un compte

Équipe

NAYME Yanis

GUEYE Babacar Mansour

CLABAU Brandon

LYOUBI Mehdi

Niveau ING3 · Enseignant : JOUD HANAY

Dépôt : https://github.com/Babacar2804/Projet-PythonSecure-log-anlysis

11/11/2026

Sommaire

1. Introduction et objectifs 3

2. Environnement et organisation du projet 3

3. Le jeu de données généré 4

4. Architecture du pipeline 5

5. Module 1 — Ingestion et parsing 7

6. Module 2 — Transformation et nettoyage 8

7. Module 3 — Agrégation et analyse 10

8. Module 4 — Moteur de règles et introspection 14

9. Performances et mémoire 16

10. Qualité du code et robustesse 17

11. Limites et améliorations possibles 17

12. Conclusion 17

1. Introduction et objectifs

L'objectif du projet est de concevoir une plateforme en ligne de commande capable d'**ingérer, nettoyer, transformer et analyser des journaux de logs système et réseau volumineux**, sans jamais charger les fichiers entiers en mémoire vive.

Pour l'éprouver, nous avons aussi écrit un **générateur** qui produit un jeu de données réaliste de **12,1 Go** (69 294 532 lignes réparties en 12 fichiers). Conformément aux consignes, ces logs sont variés (11 services, 3 formats, des incidents et des attaques simulés) et volontairement abîmés pour nécessiter un nettoyage poussé.

Le rapport suit le cheminement des données : leur génération, l'architecture du pipeline, puis les quatre modules du cahier des charges, illustrés par les résultats obtenus sur le jeu complet.

- **Module 1** — ingestion par générateurs (`yield`, `yield from`), suivi d'un flux en temps réel et parsing en dictionnaires ;
- **Module 2** — pipeline fonctionnel (`map`, `filter`, `lambda`) et routage par `match-case` ;
- **Module 3** — agrégation (`set`, `frozenset`, `defaultdict`) et détection de force brute ;
- **Module 4** — moteur de règles (`*args`, `**kwargs`) et audit par introspection.

2. Environnement et organisation du projet

2.1 Environnement virtuel et dépendances

Le projet s'exécute dans un environnement virtuel `venv` isolé. Le fichier `requirements.txt` est généré par `pip freeze` : il permet de reconstruire exactement le même environnement. Le moteur lui-même n'utilise que la bibliothèque standard ; matplotlib sert aux graphiques, reportlab à produire ce rapport, pytest et mypy à vérifier le code.

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt

python main.py demo                      # test + analyse
python main.py generate --size-gb 12     # jeu de 12 Go
python main.py analyze                   # analyse complète
python main.py analyze --limit 2000000 --level ERROR
python main.py live --duration 60        # temps réel

python -m pytest                         # 75 tests
python -m mypy                           # typage
```

2.2 Structure en package

Le code est organisé en package Python (`logengine`), un fichier par responsabilité. Le point d'entrée `main.py` pilote l'ensemble via quatre commandes. Aucun notebook n'est utilisé. Le code est versionné avec Git et partagé sur GitHub ; les données générées en sont exclues par le `.gitignore`.

Fichier	Rôle	Module
main.py	CLI : generate, analyze, demo, live ; construit le pipeline	—
generator.py	Génère le jeu de données (multiprocessus) et le flux temps réel	—
ingestion.py	Lecture paresseuse des fichiers, suivi en direct	1
parsing.py	Ligne brute → dictionnaire, routage match-case	1 et 2
transform.py	Nettoyage, normalisation, suivi qualité	2

Fichier	Rôle	Module
aggregation.py	Compteurs, ensembles, liste noire	3
detection.py	Détection de force brute	3
rules.py	Moteur de règles de sécurité	4
audit.py	Introspection globals() / locals()	4
report.py	Exports JSON / CSV et graphiques	—
tests/	75 tests pytest, un fichier par module	tous

3. Le jeu de données généré

Le générateur produit 12,1 Go de logs couvrant la période du 2026-10-01 au 2026-10-30. Chaque fichier couvre une tranche de temps (comme une rotation de logs) et est écrit par un processus séparé (`ProcessPoolExecutor`) : les 12 Go ont été produits en 75 secondes avec 7 processus.

3.1 Trois formats de lignes bien structurés

Format applicatif (~80 %) : sept champs séparés par « | ». Le dernier est un payload JSON ou une chaîne brute.

```
2026-10-01T08:15:32.481Z | INFO | api-gateway | 10.0.3.17:51432/TCP | user=u1042
  | event=http_request | {"method":"GET","path":"/api/v1/products","status":200,"latency_ms":43,...}
2026-10-01T08:15:33.017Z | WARNING | db-cluster | 10.0.100.21:6772/TCP | user=svc_db
  | event=query_slow | requête lente (2350 ms) sur la table orders
```

Format JSON (~10 %, services de paiement et utilisateurs) :

```
{"ts":"2026-10-01T08:15:32.481Z","lvl":"error","svc":"payment-service",
 "client":{"ip":"95.5.28.176","port":443,"proto":"TCP"},"user":"u9",
 "event":"payment_failed","code":502,"msg":"bank gateway error","amount":14.7,...}
```

Format pare-feu (~10 %) : champs positionnels séparés par des espaces, qui forment naturellement un tuple de connexion.

```
FW 2026-10-01T00:20:12.423Z DROP TCP 102.109.77.236 27409 10.0.100.29 22 DENY-SSH-EXT
```

3.2 Des données variées

- 11 services, 5 000 utilisateurs (distribution de Zipf : quelques comptes très actifs), environ 17 000 adresses IP (internes, bureau, publiques, IPv6) ;
- un profil d'activité réaliste : creux la nuit, pics vers 10 h et 15 h, trafic réduit de 40 % le week-end ;
- des scénarios planifiés : **pannes** de services, attaques **DDoS** par un botnet de 400 IP, **déploiements** quotidiens, **disque** qui se remplit jusqu'à saturation ;
- des attaques : **force brute** (dont certaines réussissent, suivies d'une élévation de privilèges et d'un export de données), **credential stuffing**, **scans de ports**, **injections SQL**, **XSS**, **path traversal**, outils offensifs ;
- une liste noire d'IP (fichier lui aussi imparfait) mêlant attaquants connus et IP qui n'apparaissent jamais dans les logs.

3.3 Des défauts injectés pour un nettoyage poussé

Environ 5 % des lignes sont volontairement abîmées. Certains défauts sont **réparables**, d'autres imposent un **rejet** : c'est ce que le pipeline doit distinguer.

Défaut injecté	Exemple	Traitement
Espaces, tabulations, \r	`" \t2026-… \r"`	réparé
Caractères invisibles, octets invalides, BOM	`\x00, \xff, \ufeff`	réparé
Codes couleur de terminal	`\x1b[31mERROR\x1b[0m`	réparé
Casse des niveaux / services	err, Warning, FATAL, API_GATEWAY	réparé
Format des utilisateurs	U-1042, USER_1042, user-01042, null	réparé → u1042 / anonyme
Autres formats de date	01/10/2026 08:15:32, epoch, virgule	réparé
JSON entre guillemets simples, nombres en texte	{'status': '200'}	réparé
Encodage cassé	requÃªte	réparé → requête
Latence absurde	-1, NaN, "slow"	valeur neutralisée
Lignes vides, commentaires, stack traces, doublons		supprimé
Date impossible	2026-13-45T25:61:00Z	rejeté
IP ou port impossible	999.12.4.1, 10.0.0, port 99999	rejeté
Ligne tronquée, champ manquant, JSON incomplet		rejeté

4. Architecture du pipeline

Le cœur du projet est une **chaîne de traitements paresseux**. `map()`, `filter()` et les fonctions contenant `yield` ne calculent rien tant qu'on ne leur demande pas l'élément suivant.

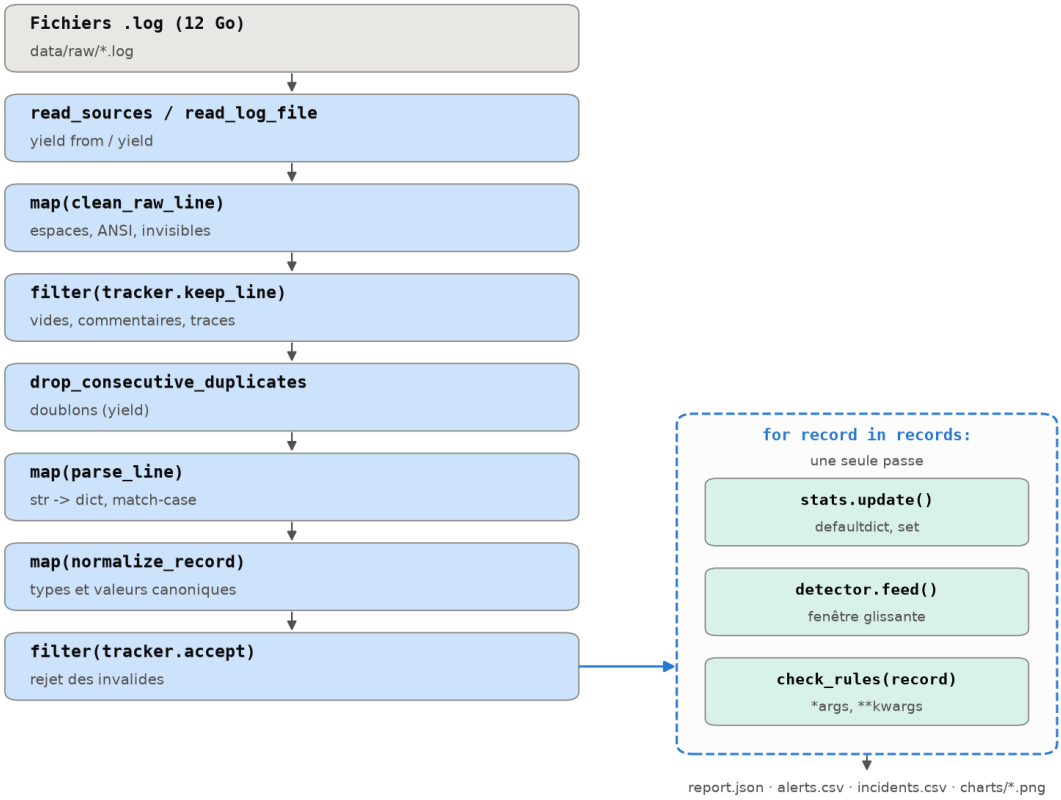


Figure 1 — Architecture du pipeline : de la lecture du disque jusqu'aux résultats.

Dans `main.py`, la construction du pipeline ne lit aucune donnée : chaque instruction enveloppe l'itérateur précédent. C'est la boucle `for` finale qui « tire » les lignes une à une à travers toute la chaîne. Une ligne est lue, nettoyée, découpée, normalisée, comptée, puis oubliée.

```
lines   = read_sources(files)                 # yield from
lines   = map(clean_raw_line, lines)          # nettoyage
lines   = filter(tracker.keep_line, lines)    # vides, #...
lines   = drop_consecutive_duplicates(lines, tracker)
records = map(parse_line, lines)              # str -> dict
records = map(normalize_record, records)      # types, casse
records = filter(tracker.accept, records)     # rejets
records = filter(lambda r: LEVEL_RANK[r["level"]] >= min_rank, records)

for record in records:                        # une passe
    stats.update(record)
    detector.feed(record)
    for alert in check_rules(record):  # check_rules = apply_security_rules(...)
        alert_writer.writerow(alert)
```

Comme un générateur ne se parcourt qu'une fois, tous les calculs (statistiques, détection, règles) sont faits **dans la même passe**. On ne conserve que des compteurs et des ensembles de taille bornée, jamais les lignes.

5. Module 1 — Ingestion et parsing

5.1 Lecture paresseuse avec yield et yield from

```
def read_log_file(path: Path) -> Iterator[str]:
    with open(path, encoding="utf-8", errors="replace", newline="") as file:
        for line in file:
            yield line

def read_sources(paths: Iterable[Path]) -> Iterator[str]:
    for path in paths:
        try:
            yield from read_log_file(path)
        except (FileNotFoundError, PermissionError) as exc:
            print(f"fichier ignoré : {path}", file=sys.stderr)
```

`yield` suspend la fonction et renvoie une ligne ; elle reprend au même endroit à l'appel suivant. `yield from` délègue à un sous-générateur : les fichiers successifs forment un flux unique. `errors="replace"` évite qu'un octet invalide ne fasse planter la lecture ; le caractère de remplacement est ensuite supprimé par le nettoyage.

5.2 Modélisation en dictionnaire et tuple de connexion

Chaque ligne devient un dictionnaire typé (`TypedDict LogRecord`) : horodatage (`datetime`), niveau, service, IP, utilisateur, évènement, payload (dict JSON ou chaîne brute), ainsi que deux listes `errors` et `fixes` qui tracent le nettoyage. Les métadonnées fixes de la connexion sont stockées dans un **tuple immuable** (`IP, Port, Protocole`) : il ne peut pas être modifié par erreur et, étant hachable, il peut servir de clé de dictionnaire ou d'élément d'ensemble (voir 7.2).

5.3 Traitement en temps réel

Le moteur sait aussi analyser un flux **en direct**. `follow_log_file` est un générateur infini qui suit un fichier en train de grossir, comme `tail -f` : quand il n'y a rien de nouveau, il attend au lieu de s'arrêter, et il n'émet une ligne qu'une fois son `\n` final écrit.

```
while deadline is None or time.monotonic() < deadline:
    chunk = file.readline()
    if not chunk:                    # rien de nouveau
        time.sleep(poll_interval)
        continue
    pending += chunk
    if not pending.endswith("\n"):  # ligne incomplète
        continue
    yield pending
    pending = ""
```

La commande `python main.py live` lance dans un `thread` un « serveur » simulé qui écrit environ 200 lignes par seconde, pendant que le programme principal suit le fichier. Les lignes passent par **exactement le même pipeline** que les archives (`build_pipeline`) : seule la source change. Extrait d'une exécution :

```
[live] 1,571 lignes lues, 1,547 valides, 1 alertes
  [HIGH    ] path_traversal   213.67.29.14    /download?file=..%2f..%2fetc%2fshadow
  [HIGH    ] ip_blacklistee   92.112.165.218  login_failed depuis une IP en liste noire
  [FORCE BRUTE] 92.112.165.218 : 5 échecs en moins de 2 min
  [HIGH    ] xss              217.135.14.178  /profile?name=javascript:alert(document.cookie)
=== Bilan temps réel : 7,065 lignes, 6,956 valides, 1 IP suspectes ===
```

6. Module 2 — Transformation et nettoyage

6.1 Routage par Structural Pattern Matching

Plutôt qu'une cascade de `if/elif`, `parse_line` décrit la **forme** attendue de chaque format. La ligne est d'abord décodée en dict (JSON), en tuple (pare-feu) ou laissée en chaîne, puis routée :

```
match decode_line(line):
    case {"ts": timestamp, "lvl": str(level), "svc": str(service),
          "client": {"ip": ip, "port": port, "proto": protocol}, **rest}:
        ...                                  # dictionnaire
    case ("FW", timestamp, action, protocol, src_ip, src_port,
          dst_ip, dst_port, *rule):
        ...                                  # tuple de connexion
    case tuple():
        return _rejected(line, "pare_feu_incomplet")
    case str() as text if text.startswith("{"):
        return _rejected(line, "json_invalide")
    case str() as text:
        return _parse_app_line(text)         # chaîne brute
```

Un motif dict vérifie la présence des clés (même imbriquées) ; un motif tuple vérifie la longueur et le premier élément ; `str(level)` contrôle le type et capture la valeur. Le même mécanisme remplace les longs `if` de normalisation des niveaux :

```
match value.strip().upper():
    case "WARNING" | "WARN" | "WRN":
        return "WARNING"
    case "CRITICAL" | "CRIT" | "FATAL" | "EMERG":
        return "CRITICAL"
```

6.2 Programmation fonctionnelle : map, filter, lambda

`map(clean_raw_line, lines)` applique la même fonction de nettoyage à chaque ligne. `filter` reçoit des **méthodes** comme prédicats (`tracker.keep_line`, `tracker.accept`) : elles décident de garder ou non l'élément et comptent au passage la cause de chaque suppression. Des `lambda` servent aux filtres de la ligne de commande et aux tris, et `filter(None, …)` élimine les lignes vides de la liste noire.

6.3 Résultats du nettoyage

69 294 532	501 369	467 239	1,4 %
lignes lues	supprimées	rejetées	taux de perte

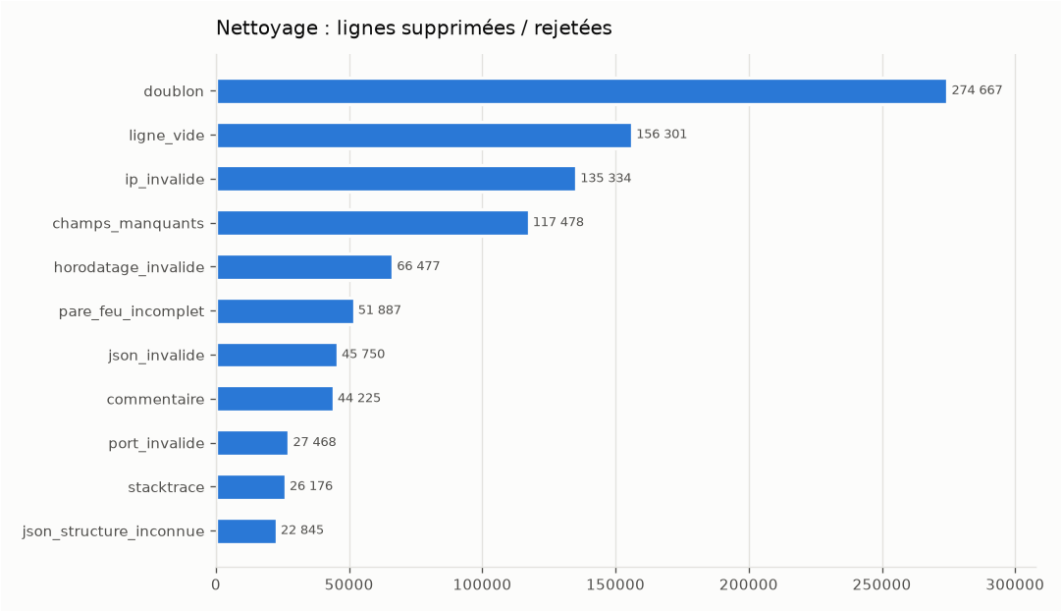


Figure 2 — Lignes supprimées avant parsing et enregistrements rejetés, par cause.

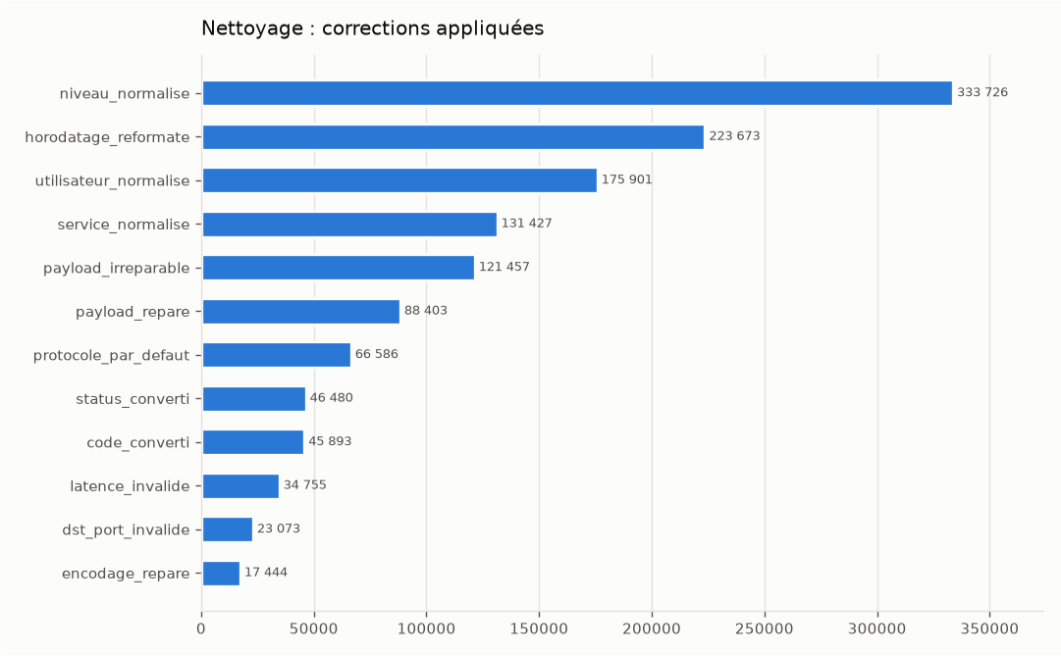


Figure 3 — Corrections appliquées : ces lignes étaient abîmées mais ont pu être récupérées.

Au total, **1 308 818 corrections** ont été appliquées. La grande majorité des défauts est donc réparée plutôt que perdue ; seules les lignes réellement inexploitables (date ou IP impossibles, ligne tronquée) sont écartées. Des exemples réels de chaque cause sont conservés dans `rejected_samples.txt`.

7. Module 3 — Agrégation et analyse

7.1 Accumulation avec defaultdict

`LogStatistics` accumule toutes les statistiques en une passe. `defaultdict(int)` crée automatiquement la valeur 0 pour une clé absente : `compteur[cle] += 1` ne lève jamais de `KeyError`. Des variantes imbriquées (`defaultdict(lambda: defaultdict(int))` pour service → niveau, `defaultdict(set)` pour IP → ports bloqués) complètent l'analyse.

68 325 924	17 538	5 009	1 081 786
logs valides	IP uniques	utilisateurs	erreurs

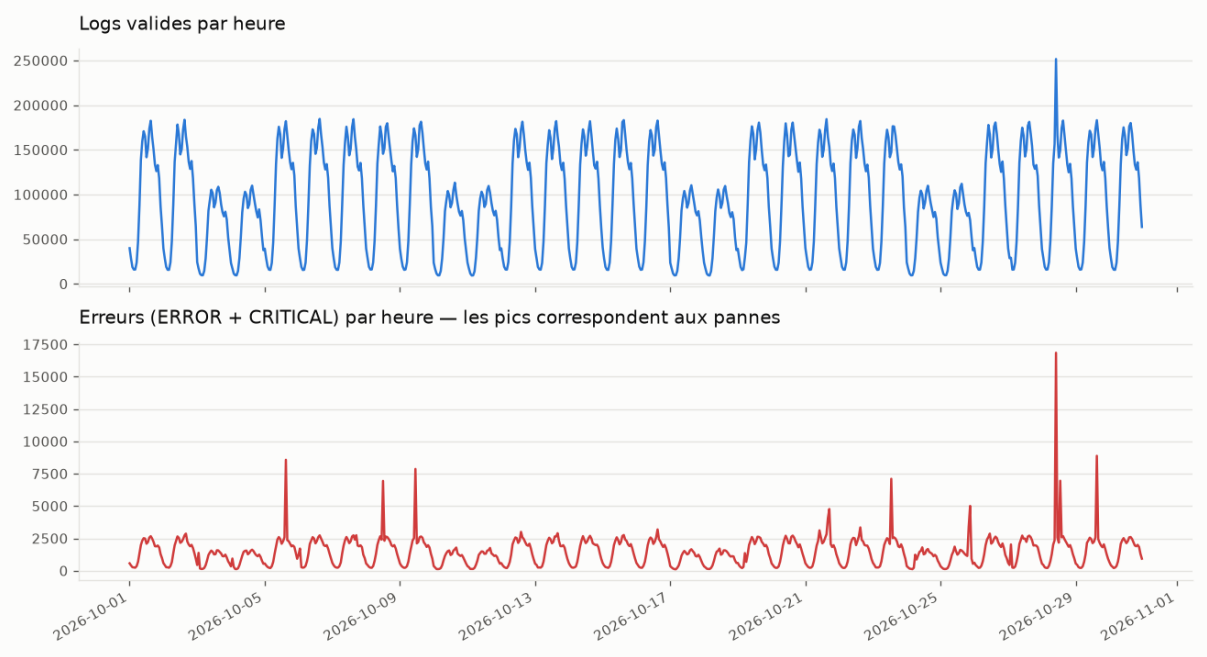


Figure 4 — Volume de logs et d'erreurs par heure. On distingue le cycle jour/nuit, les week-ends plus calmes et les pics d'erreurs des pannes simulées.

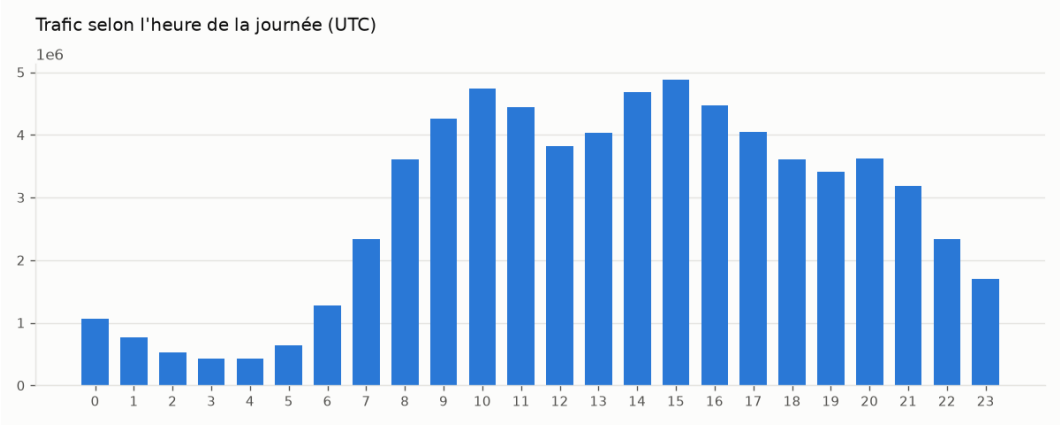


Figure 5 — Trafic selon l'heure de la journée (UTC).

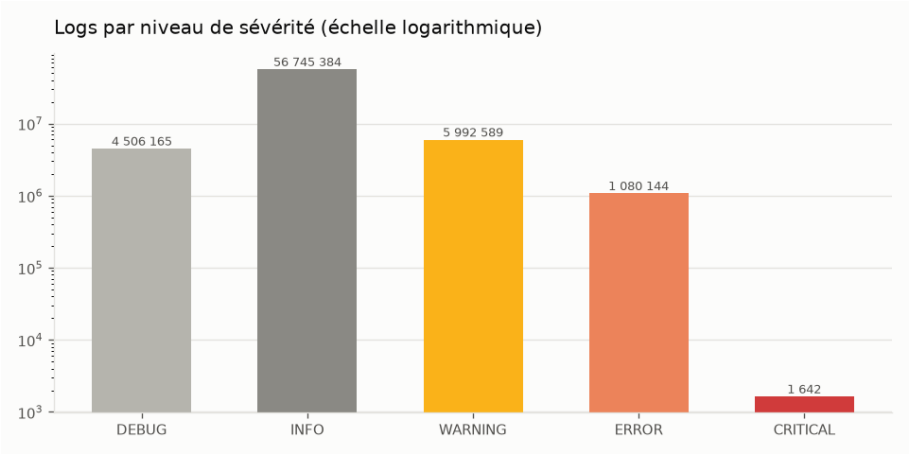


Figure 6 — Répartition par niveau de sévérité (échelle logarithmique).

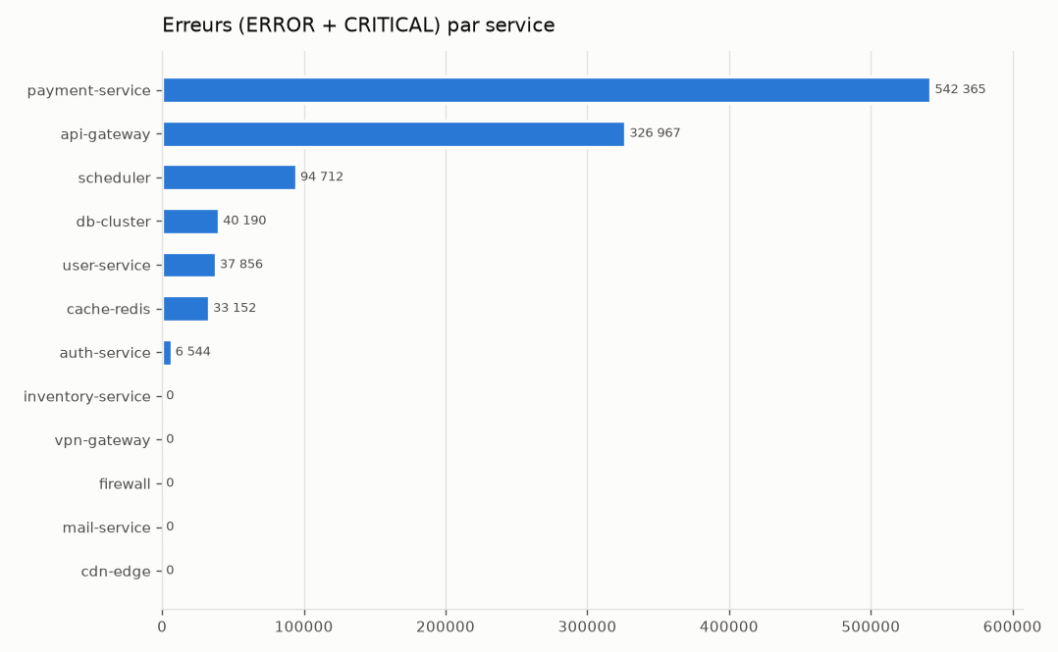


Figure 7 — Erreurs par service : les services touchés par les pannes ressortent nettement.

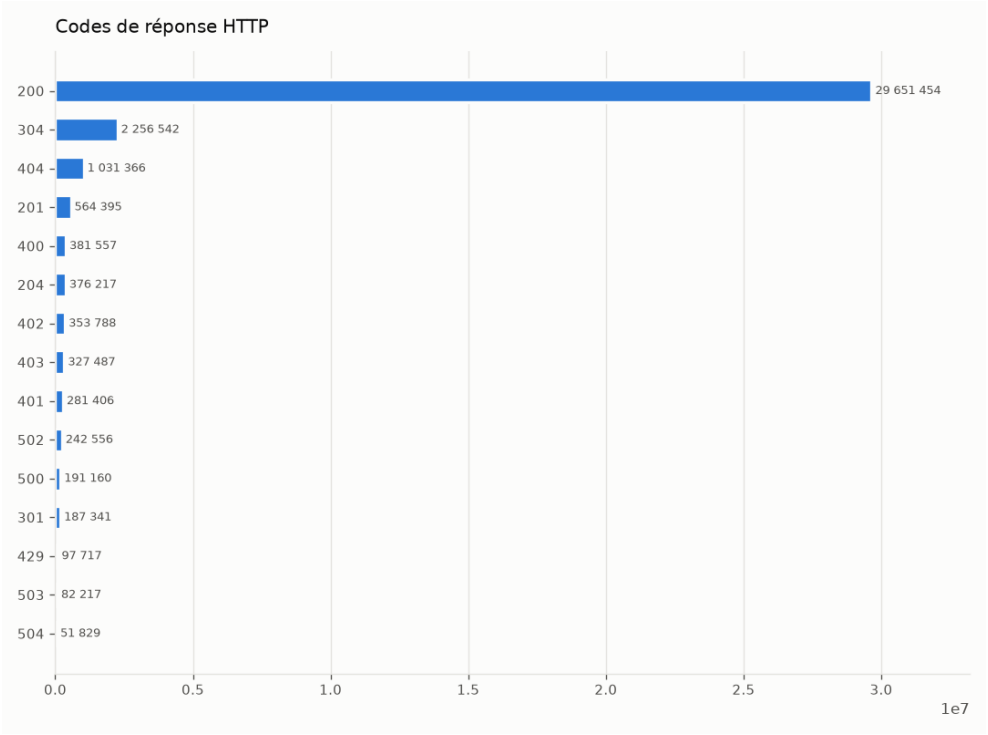


Figure 8 — Codes de réponse HTTP.

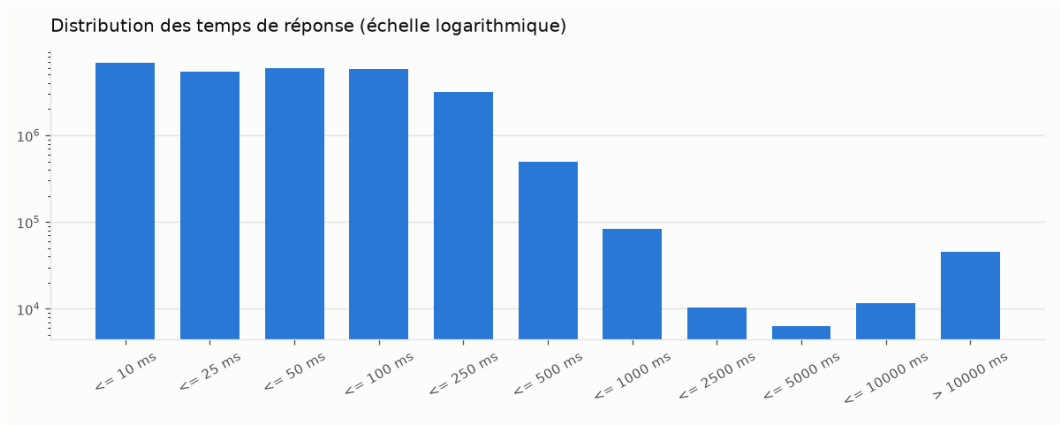


Figure 9 — Distribution des temps de réponse.

7.2 Ensembles : IP uniques et liste noire

Les IP ayant généré des erreurs sont collectées dans un `set` : les doublons sont ignorés automatiquement. La liste noire est chargée dans un `frozenset`, immuable, pour qu'elle ne puisse pas être modifiée pendant l'analyse. Les opérations ensemblistes donnent :

Opération	Signification	Nombre d'IP
actives | blacklist	union : toutes les IP connues	17 838
actives & blacklist	IP listées réellement actives	190
erreurs & blacklist	IP listées qui provoquent des erreurs	190
actives - blacklist	IP actives absentes de la liste	17 348
blacklist - actives	IP listées jamais vues (liste à mettre à jour)	300
actives ^ blacklist	différence symétrique	17 648

Sur 490 IP en liste noire, **190** sont actives dans nos logs : ce sont précisément les attaquants et machines du botnet injectés par le générateur, ce qui valide la chaîne complète.

7.3 Tuples de connexion : flux bloqués et scans de ports

Le tuple `(IP, Port, Protocole)` est exploité de trois façons. Il est d'abord **déballé** (`ip, _, protocol = record["connection"]`) pour compter les protocoles : 65 645 867 TCP, 2 659 957 UDP, 20 100 ICMP. Ensuite, pour chaque blocage du pare-feu, le tuple (IP source, port visé, protocole) sert de **clé de dictionnaire** — ce qui n'est possible que parce qu'il est immuable. 180 896 flux distincts ont été comptés ; les plus fréquents :

```
self.blocked_flows[(ip, port, protocol)] += 1

for ip, port, _protocol in self.blocked_flows:   # déballage
    ports_by_ip[ip].add(port)                    # defaultdict(set)
```

IP source	Port visé	Protocole	Blocages
141.108.253.69	8080	TCP	902
141.108.253.69	22	TCP	874
141.108.253.69	445	TCP	857
141.108.253.69	23	TCP	833
141.108.253.69	1433	TCP	833
141.108.253.69	3389	TCP	818
94.145.172.141	22	TCP	583
94.145.172.141	445	TCP	569

Enfin, en regroupant ces clés par IP (`defaultdict(set)`), on repère **30 IP** ayant tenté au moins 15 ports différents : des **scans de ports**. Dans `rule_exposed_port`, un même `match` combine un motif dictionnaire et un motif tuple (`{"connection": (str(ip), _, "TCP"), ...}`).

7.4 Détection de force brute

Pendant la lecture, `feed()` maintient pour chaque IP une **fenêtre glissante** des échecs des 120 dernières secondes (une `deque`). Une boucle `while` retire les échecs trop anciens ; au 5e échec dans la fenêtre, l'IP devient suspecte.

```
window.append((moment, user))
while window and moment - window[0][0] > self.window:
    window.popleft()
if suspect is None and len(window) >= self.threshold:
    self._suspects[ip] = {...}
```

À la fin, `summarize()` rejoue les tentatives de chaque suspect avec `for`, `continue`, `break` et la clause `else` de la boucle :

```
for moment, user, success in suspect["events"]:
    if not success:
        consecutive += 1
        last_failure = moment
        continue                  # échec : tentative suivante
    if consecutive >= self.threshold and moment - last_failure <= self.window:
        compromised_at, compromised_user = moment, user
        break                     # succès après X échecs : COMPROMIS
    consecutive = 0
else:
    status = "bloquee"          # boucle complète sans break
```

Le `else` ne s'exécute que si la boucle est allée au bout **sans** `break` : le cycle de détection est complet et aucune connexion n'a abouti, l'attaque a donc échoué.

Les incidents sont regroupés **par IP** : une même IP qui attaque plusieurs fois sur la période forme un seul incident (d'où le nombre élevé d'échecs et de comptes ciblés). Les 105 alertes d'élévation de privilèges du moteur de règles confirment, de façon indépendante, les connexions réussies après une attaque.

60	48	60
IP suspectes	IP ayant réussi	credential stuffing

IP	Statut	Type	Échecs	Comptes ciblés	Compte compromis
103.51.91.27	COMPROMIS	credential stuffing	506	132	admin
86.24.234.231	COMPROMIS	credential stuffing	500	83	svc_deploy
31.230.136.80	COMPROMIS	credential stuffing	486	119	u25
188.190.112.60	COMPROMIS	credential stuffing	484	139	admin
90.44.68.243	COMPROMIS	credential stuffing	465	145	svc_deploy
129.0.43.119	COMPROMIS	credential stuffing	460	159	root

Tableau — Exemples d'incidents détectés (incidents.csv).

8. Module 4 — Moteur de règles et introspection

8.1 Une fonction maître à arguments variables

Une règle est une simple fonction `regle(record, **config)` qui renvoie une alerte ou `None`. La fonction maître a exactement la signature demandée, `apply_security_rules(*args, **kwargs)` : `*args` regroupe un nombre variable de règles dans un tuple, `**kwargs` la configuration dans un dict. Comme l'enregistrement à analyser ne fait pas partie de ces arguments, elle **fabrique et renvoie une fonction** `check(record)`, préparée une seule fois puis appelée sur chaque ligne :

```
def apply_security_rules(*args, **kwargs):
    for rule in args:
        if not callable(rule):
            raise TypeError(...)
    disabled = kwargs.get("disabled", ())
    active = tuple(filter(lambda rule: rule.__name__ not in disabled, args))

    def check(record):
        alerts = []
        for rule in active:
            try:
                alert = rule(record, **kwargs)
            except Exception as exc:
                print(f"{rule.__name__} a échoué : {exc!r}")
            else:
                if alert is not None:
                    alerts.append(alert)
        return alerts

    return check

check_rules = apply_security_rules(*DEFAULT_RULES, blacklist=blacklist,
                                   max_latency_ms=5000)
alerts = check_rules(record)
```

Les fonctions sont manipulées comme des **objets de première classe** : rangées dans un tuple, passées en argument, **renvoyées** par une autre fonction, identifiées par `__name__`. `check` garde accès à `active` et `kwargs` après la fin de la fonction maître : c'est une **fermeture** (closure). Ajouter une règle revient à écrire une fonction ; une règle qui plante est isolée par le `try / except / else` et n'arrête pas le moteur.

Règle	Détecte	Alertes
sql injection	' OR '1'='1, UNION SELECT, DROP TABLE	9 404
xss	<script>, onerror=, javascript:	9 519
path traversal	../../etc/passwd, win.ini	9 568
scanner probe	/.env, /wp-admin, /.git/config	9 282
outil offensif	sqlmap, Nikto, Hydra, masscan	21 235
ip blacklistee	trafic d'une IP en liste noire	89 825
elevation privileges	passage d'un compte en admin	105
exfiltration donnees	export massif de données	106
requete lente	latence > 5 000 ms	56 894
port sensible expose	port 22/3389/5432 ouvert sur Internet	4 896

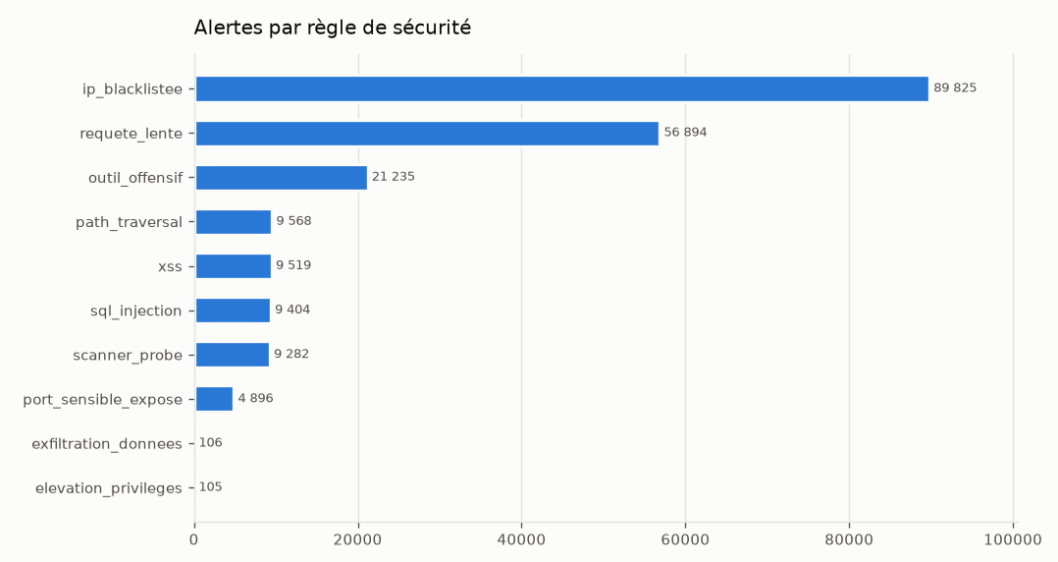


Figure 10 — Nombre d'alertes par règle de sécurité.

8.2 Audit par introspection : globals() et locals()

À la fin de l'analyse, `main.py` appelle `export_audit_report(path, globals(), locals())`. Le module d'audit décrit chaque variable (catégorie, type, taille, aperçu limité par `reprlib`) et écrit `audit.json`. C'est l'appelant qui fournit les portées : appelées dans `audit.py`, elles décriraient le module d'audit lui-même.

Variable locale	Type	Taille (octets)	Aperçu
blacklist	frozenset	32 984	frozenset({'102.12.183.222', '102.125.28.142', '102.132.58.1
check_rules	function	152	<function apply_security_rules.<locals>.check at 0x000001A50

Variable locale	Type	Taille (octets)	Aperçu
detector	BruteForceDetector	56	<logengine.detection.BruteForceDetector object at 0x000001A5
lines	generator	280	<generator object with_progress at 0x000001A50D604B80>
records	filter	48	<filter object at 0x000001A50DC3F280>
stats	LogStatistics	56	<logengine.aggregation.LogStatistics object at 0x000001A50DC
tracker	CleaningTracker	56	<logengine.transform.CleaningTracker object at 0x000001A50DC

Tableau — Extrait de audit.json (34 variables locales, 62 globales). Les itérateurs lines et records ne pèsent que quelques dizaines d'octets : ils ne contiennent pas les données.

9. Performances et mémoire

12,1 Go	17 min	69 204	75 s
analysés	durée de l'analyse	lignes / seconde	génération (7 processus)

La mémoire a été mesurée avec `tracemalloc` (option `--trace-memory`). Le pic dépend du nombre de clés distinctes (IP, utilisateurs, heures), qui est borné, et non de la taille des fichiers :

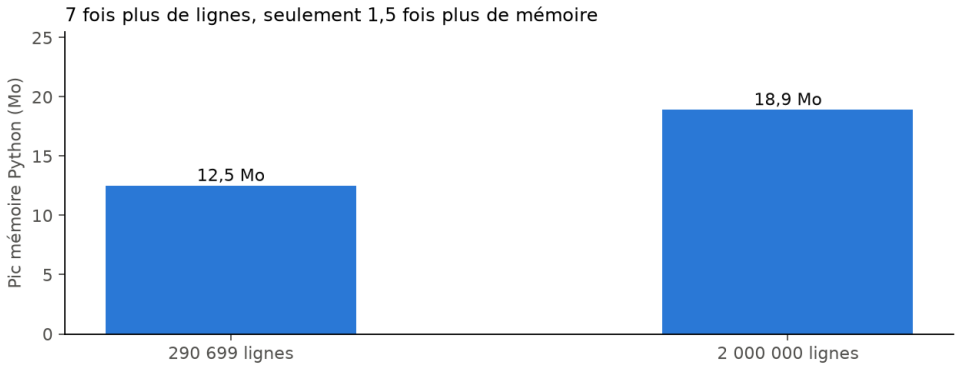


Figure 11 — Pic mémoire Python mesuré selon le nombre de lignes traitées.

Plusieurs choix vont dans ce sens : dédoublonnage par comparaison avec la seule ligne précédente (un `set` de 60 millions de lignes occuperait plusieurs Go), alertes écrites au fil de l'eau dans un CSV plutôt que gardées en liste, cache `lru_cache` pour la validation des IP (nombre d'IP distinctes limité), et raccourci `isprintable()` qui évite le nettoyage coûteux sur les lignes déjà propres.

10. Qualité du code et robustesse

- **PEP 8** : vérifié avec `pycodestyle`, aucune erreur.
- **Typage vérifié** : annotations sur toutes les fonctions, `TypedDict` pour les enregistrements et les alertes, alias `Connection = tuple[str, int, str]`. `mypy` contrôle tout le projet (code et tests) : **0 erreur**. Il avait relevé 6 imprécisions réelles, corrigées (par exemple une variable pouvant valoir `None`).
- **Tests automatiques** : **75 tests pytest**, un fichier par module : réparation des lignes sales, motifs `match`, opérations ensemblistes, sortie par `break` ou par le `else` de la boucle de détection, transmission des `**kwargs` aux règles, isolation d'une règle défaillante, suivi temps réel, et un test de bout en bout qui génère un jeu de données puis exécute tout le pipeline.
- **Exceptions** : aucune donnée sale ne fait planter le pipeline ; `try / except / else` dans le moteur de règles, gestion des fichiers manquants à l'ingestion, `ValueError` gérées pour les dates et les IP.
- **Clauses else des boucles** : dans la détection de force brute et dans l'histogramme des latences.
- **Reproductibilité** : générateur à graine fixe (`--seed`), environnement figé par `requirements.txt`, historique Git.

11. Limites et améliorations possibles

- **Analyse parallèle** : l'analyse tourne sur un seul processus (17 min pour 12,1 Go). Traiter chaque fichier dans un processus puis fusionner les compteurs diviserait ce temps par 5 à 6 sur 8 cœurs, au prix d'attaques coupées entre deux fichiers.
- **Campagnes d'attaque** : une IP qui attaque plusieurs fois sur 30 jours produit un seul incident ; on pourrait les séparer par période d'inactivité.
- **Doublons** : seuls les doublons consécutifs sont détectés.
- **Stack traces** : elles sont supprimées ; elles pourraient être rattachées à la ligne d'erreur qui les précède.

12. Conclusion

Le moteur traite 12,1 Go de logs hétérogènes et volontairement dégradés avec une empreinte mémoire de quelques dizaines de mégaoctets, grâce à une chaîne de générateurs et de fonctions d'ordre supérieur. Le nettoyage récupère l'essentiel des lignes abîmées (1 308 818 corrections) et n'écarte que les lignes inexploitables, en documentant chaque cause.

L'analyse retrouve les phénomènes injectés : cycle d'activité, pannes, DDoS, 190 IP de la liste noire actives, 60 IP menant des attaques par force brute dont 48 ont réussi, ainsi que les injections et les scans. Le moteur de règles et l'audit par introspection rendent l'ensemble extensible et observable.

Ce projet nous a permis de mettre en pratique les notions du cours (générateurs, programmation fonctionnelle, structures de données, pattern matching, arguments variables, introspection) sur un cas réaliste où la contrainte de volume impose de bien les utiliser.